

Creación de un chatbot con IA capaz de buscar información en fuentes externas

Trabajo Fin de Grado

Mario Martínez Turpín

Facultad de Informática
Universidad de Murcia

Julio 2026

Índice de la presentación

- 1 Introducción
- 2 Objetivos
- 3 Arquitectura del Sistema
- 4 Bucle de Reflexión del Agente
- 5 Evaluación y Resultados
- 6 Conclusiones

- **El auge de los LLMs:** Modelos de lenguaje como ChatGPT son excelentes interfaces conversacionales, pero sufren de *alucinaciones* cuando se les piden datos factuales y concretos.
- **La rigidez de la Web Semántica:** Wikidata y las bases RDF son extremadamente precisas, pero requieren conocer lenguajes de consulta complejos (SPARQL).
- **La Solución (KGQA):** Crear un sistema de pregunta-respuesta sobre Grafos de Conocimiento que una la accesibilidad del lenguaje natural con la precisión matemática del esquema semántico.

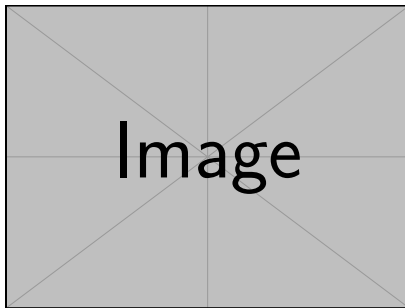
Objetivos del TFG

- ➊ Desarrollar una interfaz conversacional intuitiva y transparente.
- ➋ Traducir consultas de texto libre a código **SPARQL** válido limitando las propiedades a las permitidas en la ontología.
- ➌ Construir un **Grafo de Conocimiento local** del dominio del cine para garantizar rapidez y privacidad frente a APIs de terceros.
- ➍ Implementar un **Agente Autónomo (LangChain)** capaz de detectar y autocorregir sus propios errores de ejecución en tiempo real.

Arquitectura en 3 Capas

La arquitectura **RAG (Retrieval-Augmented Generation)** se divide en:

- **Frontend (Streamlit):** Interfaz gráfica moderna, historial de chat y panel desplegable de explicabilidad (Explainable AI).
- **Lógica y Agente (LangChain):** Cerebro del sistema, extracción de entidades y orquestación del LLM.
- **Datos (RDFLib):** Grafo local en .ttl procesado a partir de Wikidata.



Reemplazar por el diagrama de arquitectura

El Bucle de Reflexión Autocorrector

El Problema

La sintaxis SPARQL es estricta. Un LLM suele cometer errores de puntuación que provocan fallos catastróficos.

La Solución: Flujo Agéntico

- 1 El Agente genera una consulta SPARQL inicial.
- 2 El motor RDFLib intenta ejecutarla localmente.
- 3 Si hay un error, el sistema **no aborta**. Atrapa la excepción.
- 4 Se devuelve el mensaje de error de compilación al Agente.
- 5 El Agente reflexiona y regenera la consulta ya corregida de forma invisible para el usuario.

Evaluación y Resultados

Se evaluó el sistema automáticamente con un *dataset* de 20 preguntas complejas (directores, años de publicación, géneros, etc.).

Métrica Evaluada	Aciertos	Precisión (%)
Extracción y Desambiguación de Entidad	16 / 20	80.00 %
Generación de SPARQL Válido	15 / 20	75.00 %

Discusión:

- El fallo del 20 % en entidades se debe a ambigüedades idiomáticas (Ej: ".origen" vs "Inception") y los límites de peticiones HTTP.
- Un **75 % de éxito** en SPARQL demuestra que inyectar el diccionario de propiedades en el *prompt* previene alucinaciones masivas.

Conclusiones Principales:

- El uso de Agentes LangChain soluciona las barreras de uso de la Web Semántica para usuarios sin perfil técnico.
- El bucle de reflexión eleva drásticamente la tolerancia a fallos del sistema sin comprometer la latencia.

Trabajo Futuro:

- Desambiguación mediante búsqueda vectorial (*embeddings*) fonética.
- Integración de *Few-Shot Prompting* dinámico para soportar consultas *Multi-Salto* complejas.

Gracias por su atención

¿Alguna pregunta?